LSTAT2130

Introduction to Bayesian Statistics

Prof. Philippe Lambert

Academic Year : 2024 - 2025

Nathan Devaux	4085-21-00	*DATS2M*
Louis-David Piron	4933-21-00	*DATS2M*
Rémy Van Wymeersch	3601-19-00	*DATS2M*

0 Model Setup

In this project, we aim to develop a Bayesian hierarchical model to analyze the number of hospital visits y_i recorded for each individual $i \in \mathcal{I} := \{1, \ldots, n\}$, where each individual belongs to one of $\mathcal{G} := \{1, \ldots, G\}$ hospitals. The provided data $\mathcal{D}$ includes:

- $n = 919$ individuals and $G = 30$ hospitals,
- $x_{1i} \equiv$ the age of individual i,
- $x_{2i} \equiv$ a binary indicator for the presence of a chronic disease in individual i,
- $g(i) : \mathcal{I} \to \mathcal{G} \equiv$ a categorical indicator of the hospital to which individual i belongs.

The outcome y_i is assumed to follow a Poisson distribution, where the mean rate of visits, μ_i, depends on both individual covariates and a hospital-specific random effect. Specifically, the model is defined as:

$$y_i \mid \mu_i \sim \text{Poisson}(\mu_i), \quad \text{with} \quad \mu_i = v_{g(i)} \exp(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}),$$

where the random effect v_g for each hospital $g \in \mathcal{G}$ is such that $v_g \sim \text{Gamma}(\alpha, \alpha)$ and $\alpha \sim \text{Gamma}(0.01, 0.01)$ is a hyperparameter. Later on, we will refer to the $\{v_g : g \in \mathcal{G}\}$ hospitals' random effects as $\boldsymbol{v}$.

The regression coefficients $\boldsymbol{\beta} := (\beta_0, \beta_1, \beta_2)$ are assumed *a priori* to be independent and normally distributed:

$$\beta_j \sim \mathcal{N}(0, \sigma^2), \quad j = 0, 1, 2$$

with $\sigma^2 = 100$, reflecting prior uncertainty about the scale of the coefficients.

1 Question 1

(a) Likelihood Function

Let $\boldsymbol{\theta} := (\boldsymbol{\beta}, \boldsymbol{v})$ represent the parameters of our model, and α the hyperparameter. As stated earlier, we assume that $y_i \sim \text{Poisson}(\mu_i)$. For such a distribution, the probability mass function (PMF) for each observation y_i is given by:

$$P(y_i \mid \mu_i) = \frac{\mu_i^{y_i} \exp(-\mu_i)}{y_i!}$$

Since the observations are independent, the likelihood function is the product of the individual PMFs:

$$\begin{aligned}
P(\mathcal{D}|\boldsymbol{\theta}) &= \prod_{i=1}^{n} \frac{\left(v_{g(i)} \exp\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\right)^{y_i} \exp\left(-v_{g(i)} \exp\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\right)}{y_i!} \\
&= \prod_{i=1}^{n} \frac{v_{g(i)}^{y_i} \exp\left(y_i \left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\right) \exp\left(-v_{g(i)} \exp\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\right)}{y_i!} \\
&= \prod_{i=1}^{n} \frac{v_{g(i)}^{y_i} \exp\left(y_i \left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right) - v_{g(i)} \exp\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\right)}{y_i!}
\end{aligned}$$

(b) Joint Posterior

Based on Bayes' theorem, we know that the joint posterior $p(\boldsymbol{\theta}, \alpha|\mathcal{D})$ is such that :

$$p(\boldsymbol{\theta}, \alpha|\mathcal{D}) = \frac{P(\mathcal{D}|\boldsymbol{\theta}, \alpha) \times p(\boldsymbol{\theta}, \alpha)}{P(\mathcal{D})} \propto P(\mathcal{D}|\boldsymbol{\theta}, \alpha) \times p(\boldsymbol{\theta}, \alpha) = P(\mathcal{D}|\boldsymbol{\theta}, \alpha) \times p(\boldsymbol{\theta}|\alpha) \times p(\alpha)$$

We already computed the likelihood $P(\mathcal{D}|\boldsymbol{\theta}, \alpha)$ in (a), but we still need to compute the prior $p(\boldsymbol{\theta}, \alpha)$.The latter can be computed assuming *a priori* independence between the model parameters :

$$p(\boldsymbol{\theta}, \alpha) \propto \prod_{j=0}^{2} p(\beta_j) \times \prod_{g=1}^{G} p(v_g) \times p(\alpha)$$

Based on the guidelines, we know that $\beta_j \sim \mathcal{N}(0, 100)$, $v_g \sim \text{Gamma}(\alpha, \alpha)$ and $\alpha \sim \text{Gamma}(.01, .01)$. Therefore :

$$\begin{aligned}
p(\beta_j) &= \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\beta_j^2}{2\sigma^2}\right) \\
p(v_g \mid \alpha) &= \frac{\alpha^\alpha}{\Gamma(\alpha)} v_g^{\alpha-1} \exp(-\alpha v_g) \\
p(\alpha) &= \frac{b_0^{a_0}}{\Gamma(a_0)} \alpha^{a_0-1} \exp(-b_0\alpha)
\end{aligned}$$

Hence, up to a multiplicative constant, the joint prior is :

$$\begin{aligned}
p(\boldsymbol{\theta}, \alpha) &\propto \prod_{j=0}^{2} \exp\left(-\frac{\beta_j^2}{2\sigma^2}\right) \times \prod_{g=1}^{G} \frac{\alpha^\alpha}{\Gamma(\alpha)} v_g^{\alpha-1} \exp(-\alpha v_g) \times \alpha^{a_0-1} \exp(-b_0\alpha) \\
&\propto \alpha^{a_0-1} \exp\left(-\frac{\beta_0^2 + \beta_1^2 + \beta_2^2}{2\sigma^2} - b_0\alpha\right) \times \prod_{g=1}^{G} \frac{\alpha^\alpha}{\Gamma(\alpha)} v_g^{\alpha-1} \exp(-\alpha v_g)
\end{aligned}$$

and the joint posterior :

$$\begin{aligned}
p(\boldsymbol{\theta}, \alpha \mid \mathcal{D}) \propto \prod_{i=1}^{n} v_{g(i)}^{y_i} \exp\Big(y_i\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right) - v_{g(i)} \exp\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\Big) \\
\times \alpha^{a_0-1} \exp\left(-\frac{\beta_0^2 + \beta_1^2 + \beta_2^2}{2\sigma^2} - b_0\alpha\right) \times \prod_{g=1}^{G} \frac{\alpha^\alpha}{\Gamma(\alpha)} v_g^{\alpha-1} \exp(-\alpha v_g)
\end{aligned}$$

2 Question 2

Let us now identify and derive the full conditional distributions for: (a) Regression coefficients β_j, (b) the Hospital random effects v_g and (c) the hyperparameter α.

(a) Regression coefficients

Based on the the joint posterior from **Question 1**, the following joint conditional $\boldsymbol{\beta}$ can be derived by discarding constants:

$$p(\boldsymbol{\beta}|\mathcal{D}, \boldsymbol{v}, \alpha) \propto \prod_{i=1}^{n} \exp\left(y_i\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right) - v_{g(i)} \exp\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\right) \times \exp\left(-\frac{\beta_0^2 + \beta_1^2 + \beta_2^2}{2\sigma^2}\right)$$

By grouping exponents, this expression can be rewritten as :

$$p(\boldsymbol{\beta}|\mathcal{D}, \boldsymbol{v}, \alpha) \propto \exp\left(\sum_{i=1}^{n}\left(y_i(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}) - v_{g(i)} \exp(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i})\right) - \frac{1}{2\sigma^2}(\beta_0^2 + \beta_1^2 + \beta_2^2)\right)$$

This is not a known distribution

We consider now the conditional distribution of a single regression coefficient β_j for $j \in \{1, 2, 3\}$, given the data, the random effects $\boldsymbol{v}$, the hyperparameter α, and the remaining coefficients β_{-j} :

$$p(\beta_j|\mathcal{D}, \boldsymbol{v}, \alpha, \beta_{-j}) \propto \prod_{i=1}^{n} \exp\left(y_i \beta_j x_{ij} - v_{g(i)} \exp\left(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}\right)\right) \times \exp\left(-\frac{\beta_0^2 + \beta_1^2 + \beta_2^2}{2\sigma^2}\right)$$

where x_{ij} is such that $x_{i1} = 1$.

By grouping exponents and since β_j does not depend on i :

$$p(\beta_j|\mathcal{D}, \boldsymbol{v}, \alpha, \beta_{-j}) \propto \exp\left(\beta_j \sum_{i=1}^{n} y_i x_{ij} - \sum_{i=1}^{n} v_{g(i)} \exp(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i}) - \frac{\beta_j^2}{2\sigma^2}\right)$$

This is not a known distribution either.

(b) Hospital random effects

Letting $\mathcal{I}_g$ be the individuals belonging to hospital g, the conditional joint posterior can be rewritten as :

$$p(v_g|\mathcal{D}, \boldsymbol{\beta}, \alpha) \propto \left(\prod_{i \in \mathcal{I}_g} v_g^{y_i} \exp\left(-v_g \exp(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i})\right)\right) \times v_g^{\alpha-1} \exp(-\alpha v_g)$$

Letting also :

$$y_g := \sum_{i \in \mathcal{I}_g} y_i \qquad \text{and} \qquad \phi_g := \sum_{i \in \mathcal{I}_g} \exp(\beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i})$$

the conditional distribution becomes :

$$p(v_g|\mathcal{D}, \boldsymbol{\beta}, \alpha) \propto v_g^{y_g} \exp(-v_g\phi_g) \times v_g^{\alpha-1} \exp(-\alpha v_g) = v_g^{a+y_g-1} \exp(-(\alpha+\phi_g)v_g)$$

Hence,

$$v_g|\mathcal{D}, \boldsymbol{\beta}, \alpha \sim \text{Gamma}(\alpha + y_g, \alpha + \phi_g)$$

(c) Hyperparameter

The full conditional for α is proportional to:

$$p(\alpha|\boldsymbol{v}, \boldsymbol{\beta}, \mathcal{D}) \propto \alpha^{a_0-1} \exp(-b_0\alpha) \cdot \prod_{g=1}^{G} \left(\frac{\alpha^\alpha}{\Gamma(\alpha)} v_g^{\alpha-1} \exp(-\alpha v_g) \right)$$

Simplifying each component of the product :

$$p(\alpha|\boldsymbol{v}, \boldsymbol{\beta}, \mathcal{D}) \propto \alpha^{a_0-1} \exp(-b_0\alpha) \frac{\alpha^{G\alpha}}{\Gamma(\alpha)^G} \exp(-\alpha \sum_{g=1}^{G} v_g) \cdot \prod_{g=1}^{G} \left(v_g^{\alpha-1} \right)$$

Grouping terms and factorizing :

$$p(\alpha|\boldsymbol{v}, \boldsymbol{\beta}, \mathcal{D}) \propto \frac{\alpha^{a_0-1+G\alpha}}{\Gamma(\alpha)^G} \cdot \left(\prod_{g=1}^{G} v_g^{\alpha-1} \right) \cdot \exp\left(-\alpha \left(b_0 + \sum_{g=1}^{G} v_g \right) \right)$$

This is not a known distribution.

Question 3

To simulate from the joint posterior distribution of the model parameters, we construct a *Metropolis-within-Gibbs sampler* that builds upon the conditional distributions derived in **Question 2**. The sampling procedure consists of three main blocks: the hospital-level random effects $\boldsymbol{v}$, the regression coefficients $\boldsymbol{\beta}$, and the hyperparameter α.

(a) Theoretical note

Due to conjugacy, the conditional distribution of each v_g is a Gamma and can be sampled directly via *Gibbs steps*. The regression coefficients and the hyperparameter, however, do not admit closed-form conditionals, and are therefore updated using *component-wise random-walk Metropolis steps*.

In the case of $\boldsymbol{\beta}$, the log-posterior is straightforward to compute. In contrast, for the hyperparameter α, as we work log-scale $u := \log(\alpha)$, we must properly apply a change of variable.

More formally, following the transformation rule stated in Wackerly et al. (2008), if Y is a random variable with density $f_Y(y)$, and $U = h(Y)$ is a strictly monotonic transformation, then the density of U is given by:

$$f_U(u) = f_Y\left[h^{-1}(u)\right] \left| \frac{dh^{-1}(u)}{du} \right|$$

In our case, $u = \log(\alpha)$, we obtain therefore the reparametrized joint posterior :

$$p(\boldsymbol{\beta}, \boldsymbol{v}, u|\mathcal{D}) \propto p(\boldsymbol{\beta}, \boldsymbol{v}, \alpha = e^u|\mathcal{D}) \underbrace{\left|\frac{de^u}{du}\right|}_{\text{Jacobian}} = p(\boldsymbol{\beta}, \boldsymbol{v}, \alpha = e^u|\mathcal{D})e^u$$

Substituting $\alpha = e^u$ into the joint derived in **Question 2**, the unnormalized log-posterior becomes:

$$\log p(u \mid \boldsymbol{v}) \propto Ge^u u - G \log \Gamma(e^u) + (e^u - 1) \sum_{g=1}^{G} \log v_g - e^u \sum_{g=1}^{G} v_g + (a_0 - 1)u - b_0 e^u + u$$

(b) Implementation

In our implementation, regression coefficients $\boldsymbol{\beta}$ are initialized by sampling from a multivariate normal distribution centered at the posterior mode, which we obtained via numerical optimization of the unnormalized log-posterior. The covariance matrix of this proposal is taken to be the (scaled) identity, $\Sigma = k \times \mathbf{I}_3$, reflecting the assumption of prior independence across coefficients. The random hospital effects $\boldsymbol{v}$ are initialized at 1 for all 30 hospitals, hypothesizing no random effect across hospitals. The hyperparameter α is initialized by sampling $u \sim \mathcal{N}(\log 1, 0.1^2)$.

Following standard guidelines, we tune the proposal variances in the Metropolis steps to achieve optimal acceptance rates. Specifically, since we update each parameter individually (*i.e.*, component-wise), we target an acceptance rate of approximately 40%, as recommended for univariate Metropolis updates. We used normal distributions as our proposal distributions. For more details, the implementation of our MCMC algorithm is available in **Appendix C**.

(c) Convergence assessment

To first assess convergence in a naive way, we examined raw traceplots from five MCMC chains initialized near the posterior mode, as well as four chains started from deliberately extreme parameter values. These chains clearly suggest convergence to a common region of the parameter space, despite drastically different starting points.

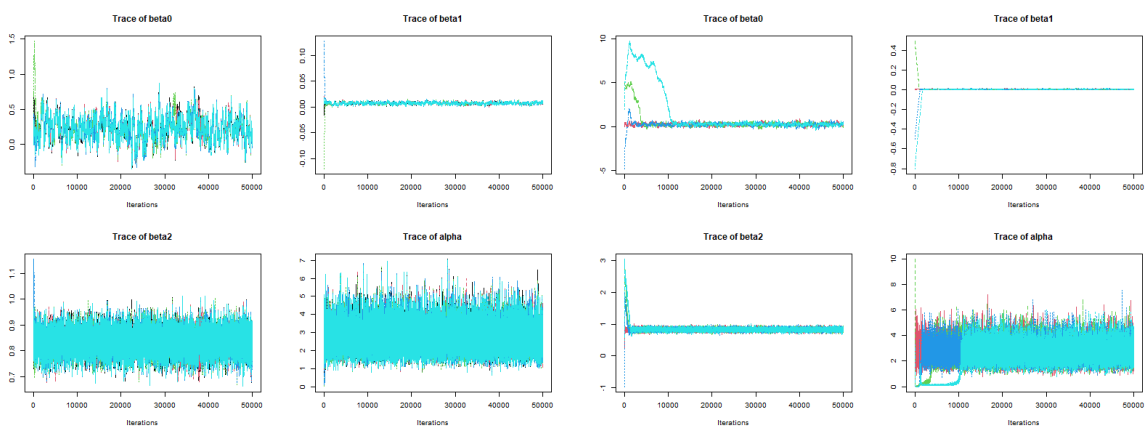

Figure 1: Traceplots of chains before burn-in. **Left**: 5 chains initialized around the posterior mode. **Right**: 4 chains initialized at extreme values.

It is also interesting to note that, overall, the further we initialize from the estimated posterior mode, the longer it takes for the chain to converge.

For those chains, the proposal variances used in the Metropolis steps were:

$$\text{sd.prop}^2_\beta = (0.062,\ 0.00115,\ 0.08), \quad \text{sd.prop}^2_u = 0.56$$

These proposals made us reach the following acceptance rate (for the 5 chains initialized around the estimated posterior mode) :

Parameter	**Chain 1**	**Chain 2**	**Chain 3**	**Chain 4**	**Chain 5**
β_0	0.376	0.376	0.377	0.376	0.375
β_1	0.391	0.391	0.394	0.393	0.391
β_2	0.410	0.410	0.412	0.410	0.409
α	0.458	0.456	0.458	0.456	0.456

Table 1: Acceptance rates for each parameter across the 5 MCMC chains.

In order to make inference, we need samples from a stationary posterior distribution. To ensure this, we must determine an appropriate burn-in period. The goal of burn-in is to remove a large number of early samples that are still strongly influenced by initial conditions, and thus highly serially correlated. As a first intuitive diagnostic, we examine the ACF of Chain 1. To make this assessment more robust, we also compute the Gelman-Rubin shrink factor over the early iterations of our 5 chains. As shown in Figure 2, the ACF reveals autocorrelation at early lags. Additionally, the Gelman–Rubin statistic computed over the first 10,000 iterations (Figure 3) shows $\hat{R} > 1.1$ for several parameters, indicating that convergence has not yet been reached. These two diagnostics together support our choice of a burnin of around 5,000 iterations.

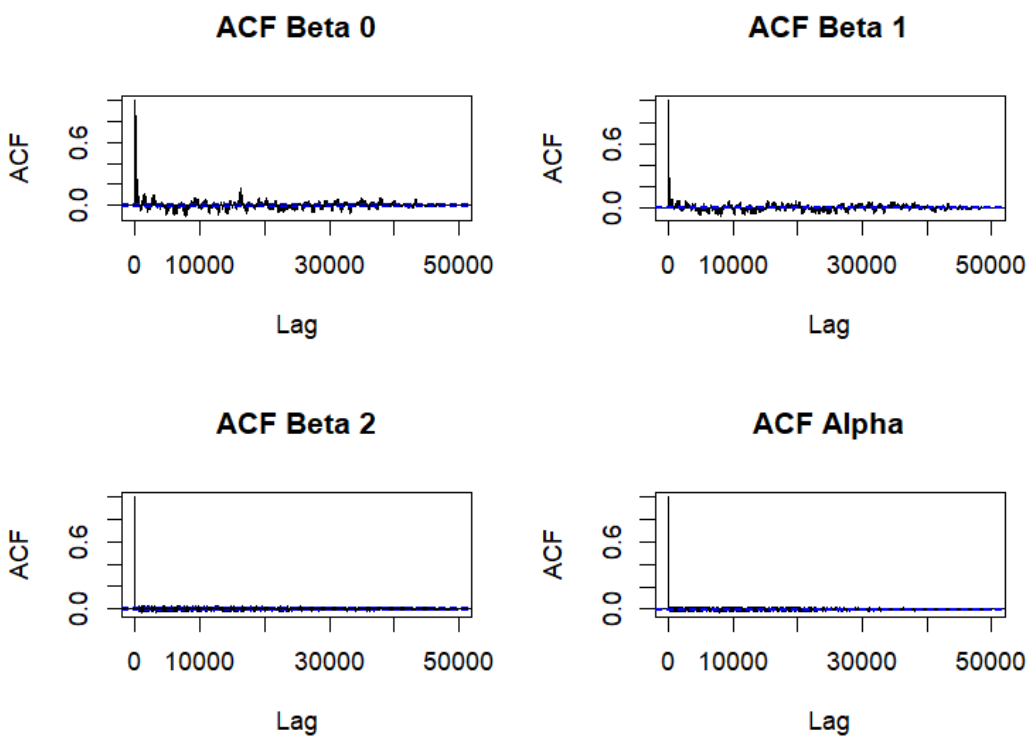

Figure 2: Autocorrelation plots for the first MCMC chain. All parameters exhibit significant autocorrelation at early lags, motivating a burn-in.

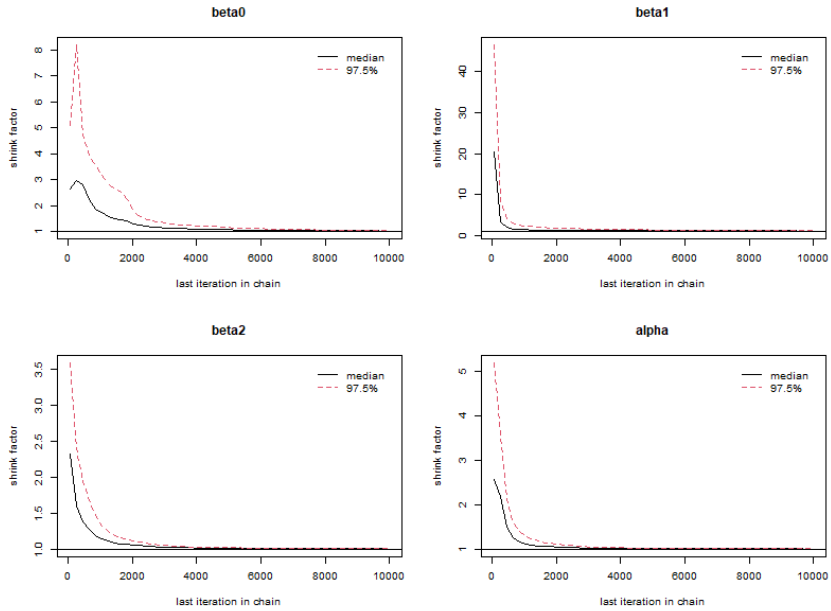

Figure 3: Gelman-Rubin shrink factor for the first 10,000 iterations. Convergence is not yet achieved for all parameters.

After burn-in, the chains appear to mix well and stabilize. This is further strengthened by the Gelman-Rubin diagnostic, which evaluates the convergence of the chains via the potential scale reduction factor (PSRF). As shown in **Appendix A**, all parameters have a PSRF point estimate and upper confidence interval equal to 1. This indicates that the between-chain and within-chain variances are indistinguishable, and therefore suggests that convergence has been achieved. The multivariate PSRF also equals 1.

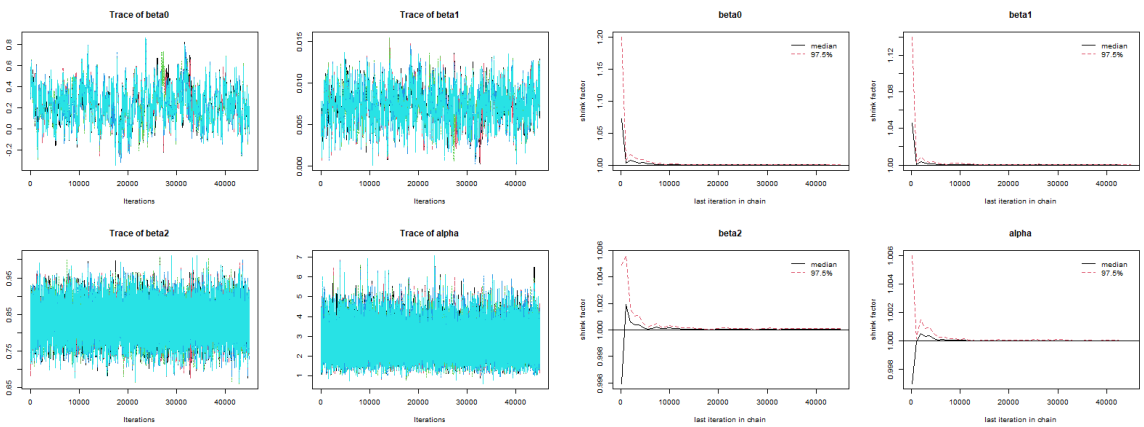

Figure 4: Traceplots of chains before and Gelman-Rubin shrink factor after burnin

To assess within-chain stationarity, we also applied the Geweke diagnostic to each chain individually. Most segments show Z-scores within the $[-1.96, 1.96]$ confidence bounds, suggesting again that convergence was reached.

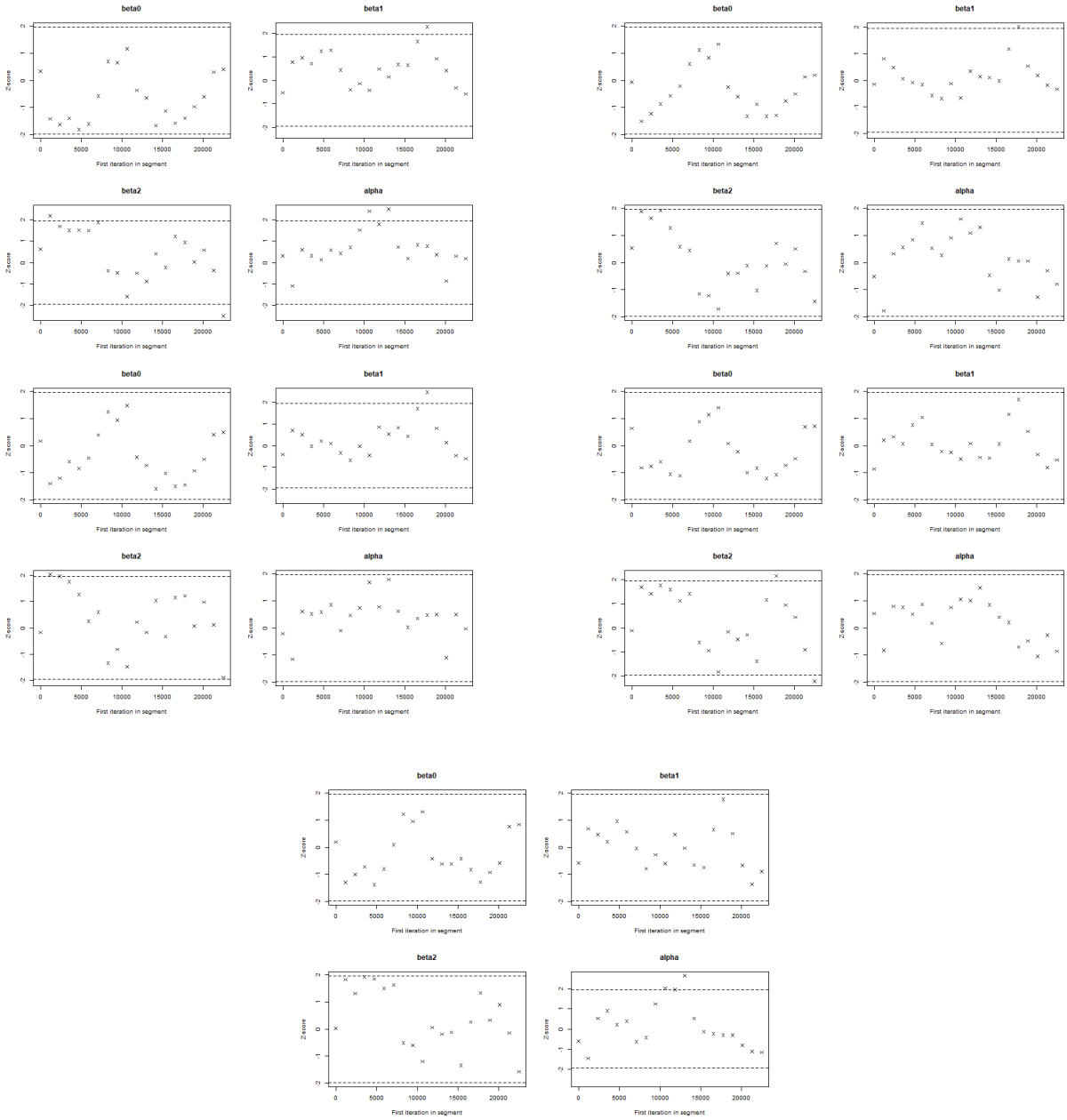

Figure 5: Geweke diagnostic plots (Z-scores by segment).

Another important aspect for inference is the effective sample size (ESS) for each parameter. The ESS quantifies the amount of independent information contained in the MCMC samples after accounting for autocorrelation.

Parameter	β_0	β_1	β_2	α
ESS	749.5	1631.6	24947.6	39157.2

Table 2: Effective sample sizes (ESS) for each parameter after burn-in.

All parameters have sufficiently large effective sample sizes to ensure reliable inference. Yet, β_0 has a comparatively smaller ESS, which needs to be taken into account.

Question 4

(a) Summary of our model parameters

Thanks to the results obtained in question 3, we are able to give detailed numerical summaries of the posterior distributions for our model parameters.

Parameter	Mean	SD	95% HPD interval	95% CI
β_0	0.229	0.155	$[-0.077,\ 0.533]$	$[-0.074,\ 0.537]$
β_1	0.00726	0.00205	$[0.00325,\ 0.0112]$	$[0.00331,\ 0.0113]$
β_2	0.835	0.0425	$[0.750,\ 0.915]$	$[0.752,\ 0.918]$
α	2.89	0.75	$[1.495,\ 4.383]$	$[1.607,\ 4.556]$

Table 3: Posterior summaries for model parameters and hyperparameter α.

Based on Table 3, we observe that both β_1 and β_2 appear to have a positive effect on the number of hospital visits. Moreover, the similarity between the HDP intervals and the quantile-based credible intervals suggests that the posterior distributions of the β_j parameters are approximately symmetric (see Figure 12 for visual confirmation).

Note as well that each coefficient β_j can be interpreted through $\exp(\beta_j)$, which gives the multiplicative change in the mean number of hospital visits for individual i associated with a one-unit increase in x_{ij}, keeping all other variables constant.

Moreover, the hyperparameter α has a posterior mean of 2.89 and a standard deviation of 0.75. We also note that the HPD intervals and the quantile-based credible intervals are not identical, which indicates that the posterior distribution of α is not perfectly symmetric. This asymmetry is also visually apparent in Figure 12.

We now take a look at the hospitals random effects v_g :

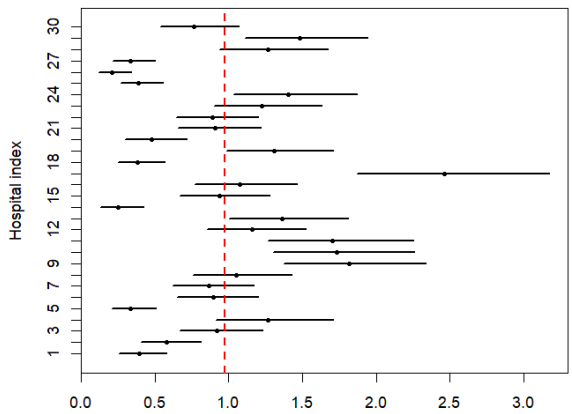

Figure 6: 95% credible intervals for hospital random effects

As shown in Figure 6, the hospital-specific random effects fluctuate between 0.1 and 3.5. Note as well that the median (red dashed line) equals 1, indicating that 50% of the hospitals have random effects smaller than 1, which reduces the number of visits compared to the average, while the other 50% have random effects greater than 1, which increases

the number of visits compared to the average.

The width of the interval reflects the level of uncertainty: the wider the interval, the less precise the estimate. The position of the interval relative to the red line provides interpretative insight: if the interval lies predominantly to the right of the red line (*e.g.* hospital 17), the hospital tends to have a positive effect on the number of visits (*i.e.*, higher than median expected hospital visits), whereas intervals lying to the left indicate a reducing effect on visit frequency (*e.g.* hospital 14). Note as well that many hospitals have a CI covering 1, suggesting that their effect on the number of hospital visits might be either increasing or decreasing.

(b) Posterior predictive performance

Prediction

In order to make predictions, we start from the stationary sample of the joint posterior $p(\boldsymbol{\beta}, \boldsymbol{v}, \alpha \mid \mathcal{D})$ generated in the previous step (Question 3). For each posterior sample of the parameters, we calculate the expected rate μ_i for each individual and then sample the corresponding number of hospital visits $\tilde{y}_i$ from a Poisson distribution with mean μ_i.

By doing so, we obtain the following predicted distribution (in orange) :

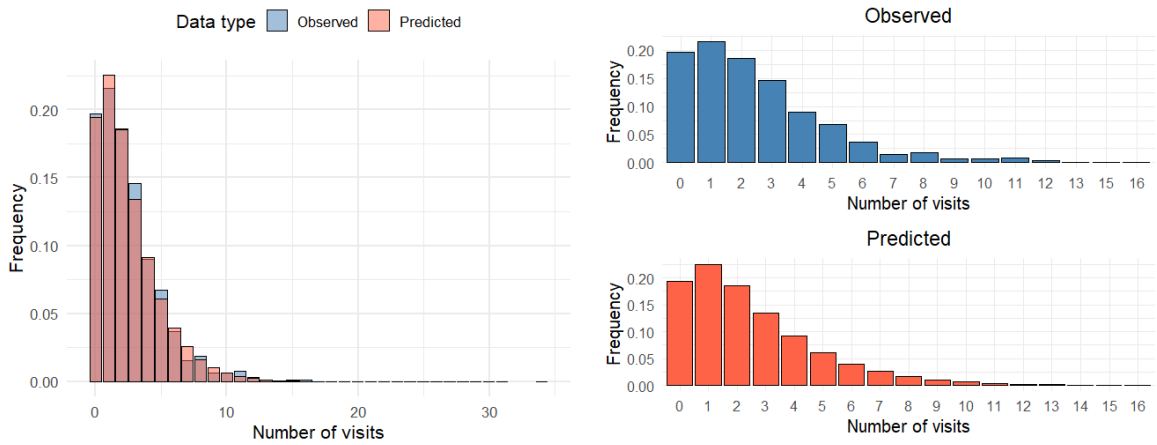

Figure 7: Distribution of the observed data (blue) and predicted data (orange).

The observed and predicted distribution really look alike, indicating that the model is effectively estimating the number of hospital visits and is therefore valid. This idea is further strengthened when we look at the following summary statistics :

	Mean	SD	Q1	Median (Q2)	Q3	IQR
Observed	2.4951	2.3932	1	2	4	3
Predicted	2.4950	2.3762	1	2	4	3

Table 4: Numerical summaries of observed and predicted distributions (transposed).

Indeed, the predicted distribution is really close to the observed distribution, in terms of central tendency (mean and median) and spread (standard deviation and interquartile range).

Question 5

In this section, our objective is to re-implement the MCMC model using `JAGS`, in order to compare the posterior results obtained in `R` with those generated by `JAGS`.

Parameter	Mean	SD	95% CI
β_0	0.2292	0.1602	$[-0.079, 0.550]$
β_1	0.0074	0.0021	$[0.0032, 0.0116]$
β_2	0.8353	0.0424	$[0.752, 0.918]$
α	2.8877	0.7571	$[1.633, 4.574]$

Table 5: Posterior summary of parameters with 95% credible intervals

Fortunately, as shown in Table 5, our results are very close to those obtained in Table 3. We can draw the same conclusions as before, which reinforces our confidence in the correctness of our MCMC implementation in R.

The same goes for the hospital random effects, very close to the ones obtained before :

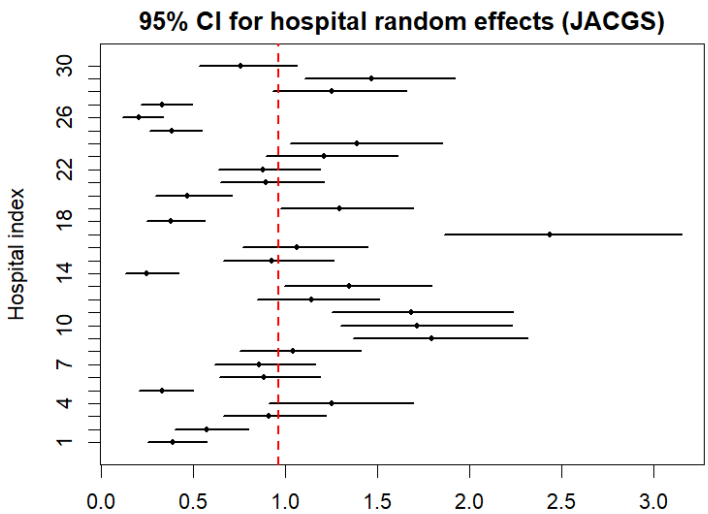

Figure 8: 95% credible intervals for hospital random effects (using JAGS code)

Question 6

In this section, we would like to evaluate robustness of our model to the following changes:

$$\begin{aligned} v_g &\sim \text{Gamma}(\alpha, \alpha) & \longrightarrow \quad & \log(v_g) \sim \mathcal{N}(0, \tau^{-1}) \\ \alpha &\sim \text{Gamma}(a_0, b_0) & \longrightarrow \quad & \tau \sim \text{Gamma}(a_0, b_0) \end{aligned}$$

Letting $z_g = \log(v_g) \iff v_g = \exp(z_g)$, our Poisson regression model becomes :

$$y_i \mid \mu_i \sim \text{Poisson}(\mu_i) \quad \text{with} \quad \mu_i = \exp(z_{g_{(i)}} + \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i})$$

Recall that $v_g = \exp(z_g)$ is seen as a hospital-specific random effect/intercept, accounting for the influence of hospital g. To understand how this random intercept v_g acts in our model, let us remember that β_0 is a fixed intercept. Let us also remember that, when x_{i1} and $x_{i2} = 0$, and when $v_g = 1$ (no random effect), $\mu_i = \exp(\beta_0)$, *i.e.* the mean number of visits without covariates and without considering a hospital-specific random effect. Incorporating the random effect v_g from hospital g, the mean becomes $\mu_i = v_g \exp(\beta_0)$, allowing each hospital to have its own base level (*i.e.* we allow for heterogeneity).

As in Section 5, we now use `JAGS` to sample from the posterior distribution. The obtained numerical summary is available in the following Table :

Parameter	**Mean**	**SD**	**95% HPD interval**	**95% CI**
β_0	0.0404	0.163	$[-0.28,\ 0.363]$	$[-0.281,\ 0.362]$
β_1	0.0074	0.002	$[0.0033,\ 0.0115]$	$[0.00328,\ 0.0115]$
β_2	0.836	0.042	$[0.752,\ 0.918]$	$[0.753,\ 0.918]$
τ	2.412	0.683	$[1.178,\ 3.778]$	$[1.28,\ 3.945]$

Table 6: Posterior summaries for model parameters and hyperparameter τ with `JAGS`.

Compared to the previous model with $v_g \sim \text{Gamma}(\alpha, \alpha)$ and $\alpha \sim \text{Gamma}(a_0, b_0)$, a few changes are to be noticed. Indeed, the coefficient β_0 shows a relatively small mean (0.0416) in comparison with the initial model, which had a mean of 0.2292. Nevertheless, the former still has a slight positive impact on the number of visits. Moreover, note that 0 lies both in its HPD and quantile-based credible interval, as within our previous summaries. On the other hand, β_1 and β_2 seem to be robust to a change in prior, since their summary statistics are very similar to the one in Section 5. Finally, both 95% HPD and quantile-based intervals are still very similar for each β_j, suggesting that their posterior distributions are approximately symmetric.

On the other hand, the new hyperparameter τ has also a relatively smaller mean compared to the previous hyperparameter α (2.887), which is probably explained by the change in prior. However, α and τ are not drastically different (see Figure 9).

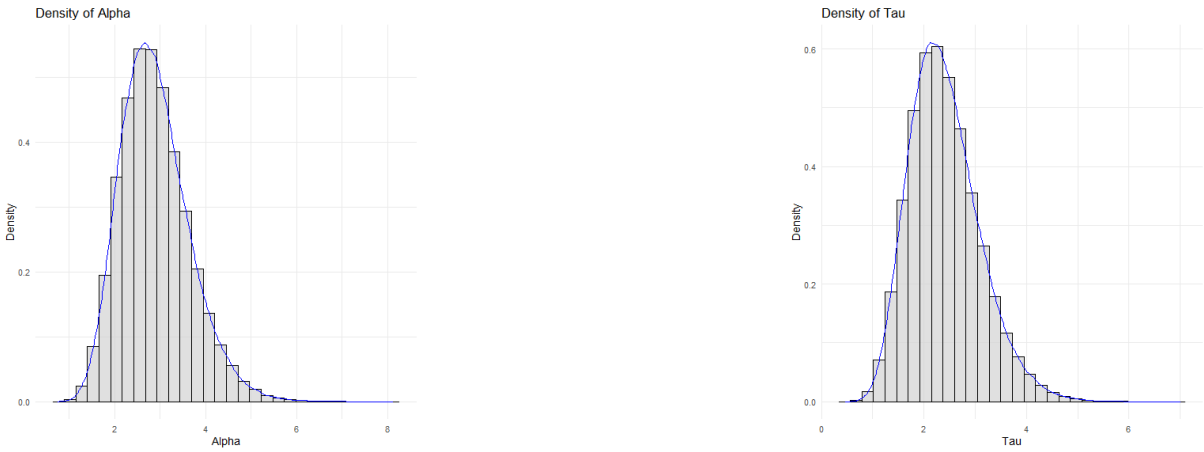

Figure 9: Density plot of α (left) and τ (right)

As far as the hospital-specific random effects v_g are concerned, let us show the different 95% credible intervals for each of them :

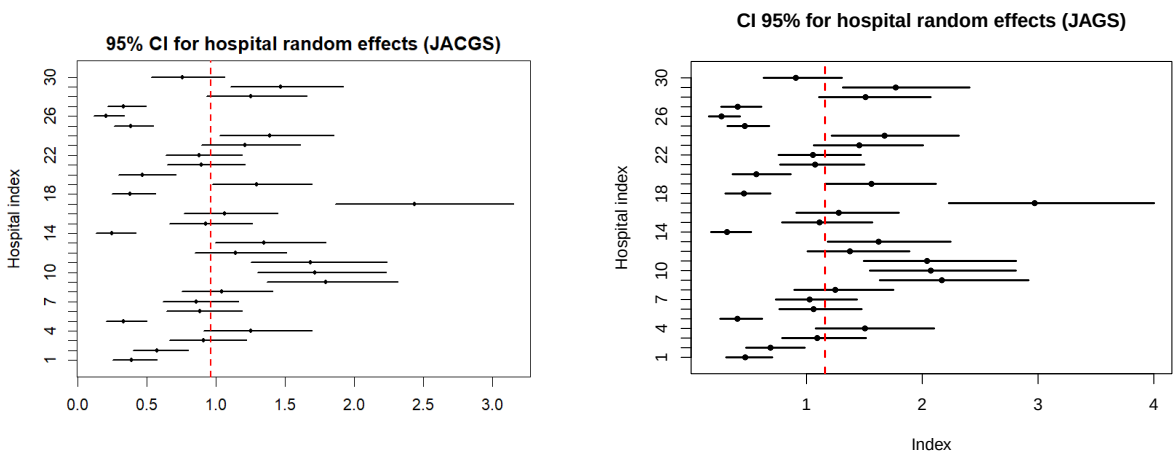

Figure 10: 95% credible intervals for hospital random effects. **Left :** Gamma prior on v_g. **Right** : Normal prior on $\log v_g$.

Interestingly, the hospital-specific random effects now range approximately between 0.2 and 4, with a higher posterior median (around 1.2), compared to a range of 0.1 to 3.5 and a lower median (0.98) in the previous implementation. Despite these shifts, the relative ranking of the hospitals remains unchanged. In the following, we try to intuitively motivate this shift.

For the gamma prior, $v_g \sim \text{Gamma}(\alpha, \alpha)$, $\mathbb{E}[v_g] = \alpha/\alpha = 1$ by construction, *a posteriori*, $\mathbb{E}[v_g \mid \mathcal{D}] \approx 1$ and we observe that the posterior median is a bit smaller than the posterior mean. However, with $\log(v_g) \sim \mathcal{N}(0, \tau^{-1})$ prior, it appears that $\mathbb{E}[v_g \mid \mathcal{D}] > 1$:

$$\mathbb{E}[v_g \mid \mathcal{D}] \approx \frac{1}{M} \sum_{m=1}^{M} \exp(z_g^{(m)}) \approx 1.205$$

where $z_g^{(m)}$ denotes the value of the random effect z_g at iteration m of the MCMC sampling. Moreover, with this new prior on $\log(v_g)$, the posterior median is around 1.16, which is again a bit smaller than the posterior mean, revealing a slight right-skewed distribution.

It now makes sense to observe that the posterior mean of β_0 is significantly smaller with this new prior. Indeed, since the posterior mean for v_g is now approximately 1.205 compared to ≈ 1, the predicted μ_i tends to be higher given this change in posterior mean for v_g, *ceteris paribus*. To maintain consistency in the observed data, the model has to estimate a lower fixed intercept β_0 to counterbalance this upward shift in μ_i.

Let us now compare the distributions of observed y_i and predicted $\tilde{y}_i$ by our new model in Figure 11. Graphically, their distributions are closely overlapping, which suggests that our model captures well the structure and the variability in the data. Therefore, one may claim that the posterior predictive distribution is adequate. While the prior on v_g differs for the two models, the posterior predictive distributions of y_i are very similar.

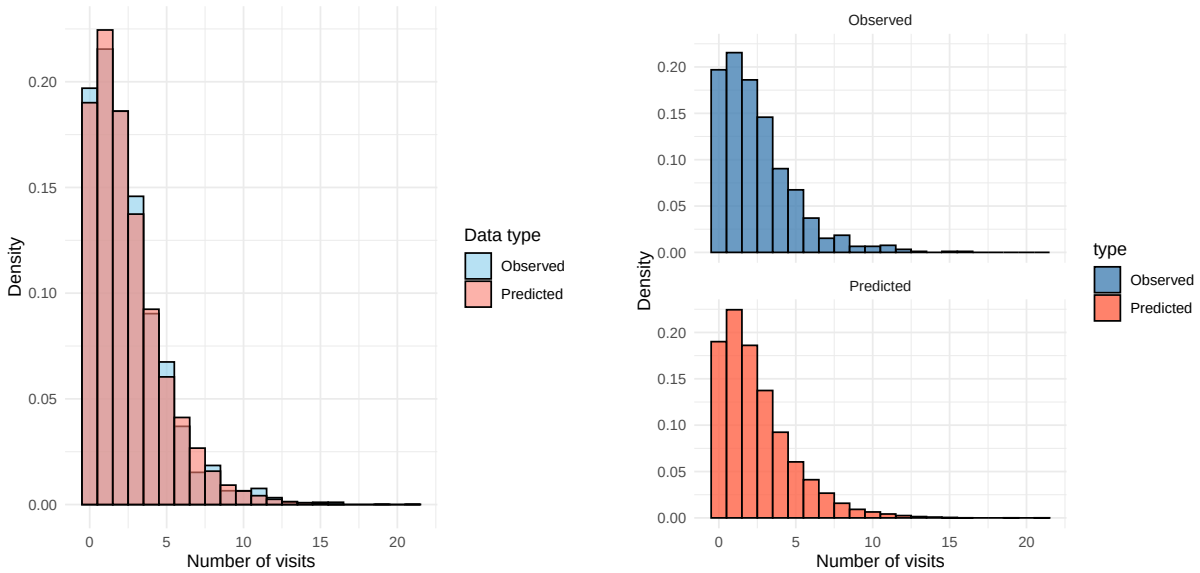

Figure 11: Distribution of the observed data (blue) and predicted data (orange).

A Gelman-Rubin $\hat{R}$

Parameter	Point Estimate	Upper C.I.
β_0	1.000	1.000
β_1	1.000	1.000
β_2	1.000	1.000
α	1.000	1.000
Multivariate PSRF	1.000	

Table 7: PSRF computed via the Gelman–Rubin diagnostic.

B Densities of main model parameters

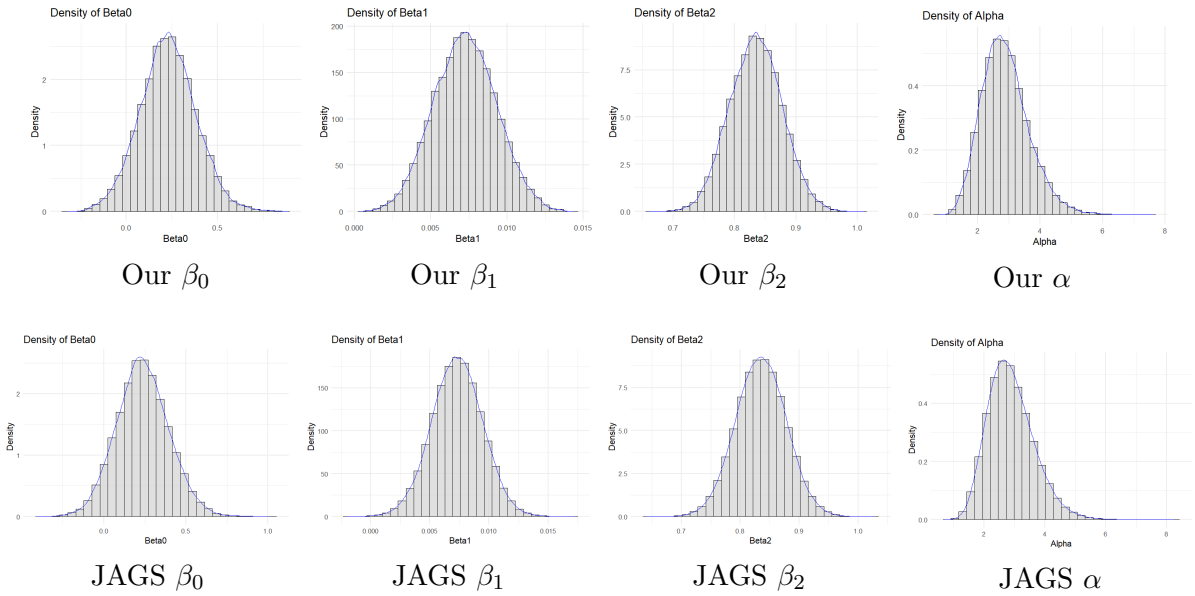

Figure 12: Density plots for the parameters β_0, β_1, β_2, and α. The first row corresponds to Our Implementation, and the second row to JAGS.

C Final R Code

```r
##########################################################################
## Project: Introduction to Bayesian statistics
##########################################################################

# --- Data import ---
HospitalVisits <- read.delim("HospitalVisits.txt")
y <- HospitalVisits$y
x1 <- HospitalVisits$age
x2 <- HospitalVisits$chronic
hospital <- HospitalVisits$hospital
G <- 30

# --- Libraries needed ---
library(mvtnorm)
library(coda)
library(ggplot2)
library(dplyr)
library(coda)
library(patchwork)

##########################################################################
# QUESTION 3 ------------------------------------------------------------#
##########################################################################

# --- Log-likelihood and Log-posterior ---

# Log-likelihood function for Poisson distribution
loglik <- function(beta, y, x1, x2, hospital, v) {
  mu <- v[hospital] * exp(beta[1] + beta[2]*x1 + beta[3]*x2)
  ll <- sum(dpois(y, mu, log = TRUE))
  return(ll)
}

# Log-posterior function for beta
logpost_beta <- function(beta, y, x1, x2, hospital, v, sigma2 = 100) {
  ll <- loglik(beta, y, x1, x2, hospital, v)  # Likelihood
  lp <- sum(dnorm(beta, 0, sqrt(sigma2), log = TRUE))  # Log-prior for beta
  return(ll + lp)
}

# Log-posterior function for alpha
logpost_alpha <- function(u, v, a0 = 0.01, b0 = 0.01) {
  # Reparametrize alpha -> u = log(alpha)
  alpha <- exp(u)
  G <- 30
  # Log-posterior for alpha with the Jacobian adjustment
  lp <- G * alpha * u - G * lgamma(alpha) +
    (alpha - 1) * sum(log(v)) - alpha * sum(v) +
    (a0 - 1) * u - b0 * alpha + u
  return(lp)
}

# --- Posterior Estimation (Laplace Approximation) ---

# Used to find the posterior mode using optim
find_posterior_mode <- function(init, y, x1, x2, v, hospital) {
  # Negative log-posterior function for optimization
```

```r
  neg_log_post <- function(beta) -logpost_beta(beta, y, x1, x2, hospital, v)
  opt <- optim(init, neg_log_post, hessian = TRUE)
  return(list(mode = opt$par, Sigma = solve(opt$hessian)))
}

init <- c(0, 0, 0)
opt <- find_posterior_mode(init, y, x1, x2, v = rep(1, 30), hospital)
beta_mode <- opt$mode
Sigma <- opt$Sigma

# Function to initialize the chains from a multivariate normal distribution

starting_values <- function(starting_index,scale = .01){
  set.seed(123)
  list(
    beta = rmvnorm(5,beta_mode,scale*diag(nrow = 3))[starting_index,],
    u = rnorm(1, mean = log(1), sd = 0.1),
    v = rep(1, 30)
  )
}


# ---MCMC SAMPLER ---

run_mcmc <- function(starting_values, M=20000, y, x1, x2, hospital, sd_beta = c(1, 1, 1), sd_u = 1) {
  # Starting values for the parameters
  beta <- starting_values$beta
  u <- starting_values$u        # Initial alpha (log-scale)
  alpha <- exp(u)               # Convert to alpha
  v <- starting_values$v

  G <- 30
  n <- length(y)

  # Output storage for posterior samples
  beta_post <- matrix(NA, M, 3)
  alpha_post <- numeric(M)
  v_post <- matrix(NA, M, G)

  # Acceptance rates
  accept_beta <- rep(0, 3)
  accept_alpha <- 0

  # MCMC sampling loop
  for (m in 1:M) {
    # ---- 1. Sample v_g | y, beta, alpha [Gibbs] ----
    for (g in 1:G) {
      idx <- which(hospital == g)
      y_g <- sum(y[idx])
      phi_g <- sum(exp(beta[1] + beta[2] * x1[idx] + beta[3] * x2[idx]))
      v[g] <- rgamma(1, shape = alpha + y_g, rate = alpha + phi_g)
    }

    # ---- 2. Sample beta | y, v, alpha [Metropolis component-wise] ----
    for (j in 1:3) {
      beta_prop <- beta
      beta_prop[j] <- rnorm(1, beta[j], sd_beta[j])                # Propose new value for beta_j
      lp_prop <- logpost_beta(beta_prop, y, x1, x2, hospital, v)    # Log-posterior for proposed beta
```

```r
      lp_curr <- logpost_beta(beta, y, x1, x2, hospital, v)         # Log-posterior for current beta
      prob <- min(1, exp(lp_prop - lp_curr))
      if (runif(1) < prob) {                                        # Accept the proposal
        beta[j] <- beta_prop[j]
        accept_beta[j] <- accept_beta[j] + 1
      }
    }

    # ---- 3. Sample alpha | v [MH on log scale] ----
    u_prop <- rnorm(1, u, sd_u)                                     # Propose new alpha (log scale)
    lp_prop <- logpost_alpha(u_prop, v)                             # Log-posterior for proposed alpha
    lp_curr <- logpost_alpha(u, v)                                  # Log-posterior for current alpha
    prob <- min(1, exp(lp_prop - lp_curr))                          # Acceptance probability
    if (runif(1) < prob) {                                          # Accept the proposal
      u <- u_prop
      alpha <- exp(u)                                               # Convert back to alpha
      accept_alpha <- accept_alpha + 1
    }

    beta_post[m, ] <- beta
    alpha_post[m] <- alpha
    v_post[m, ] <- v
  }

  cat("Acceptance rates:\n")
  cat("  beta:", round(accept_beta / M, 3), "\n")
  cat("  alpha:", round(accept_alpha / M, 3), "\n")

  list(beta_post = beta_post, alpha_post = alpha_post, v_post = v_post)
}



# Preliminary investigation of Chain 1 ----------------------------------------#

M <- 50000
sd_beta <- c(0.062, 0.00115, 0.08)
sd_u <- 0.56
start <- starting_values(starting_index = 1)

res <- run_mcmc(start,M, y, x1, x2, hospital,sd_beta,sd_u)

# ACF of Chain 1 (without burnin)
burnin <- 0
beta_post <- res$beta_post[-(1:burnin), ]
v_post <- res$v_post[-(1:burnin), ]
alpha_post <- res$alpha_post[-(1:burnin)]

mcmc_beta <- mcmc(beta_post)
mcmc_alpha <- mcmc(alpha_post)

par(mfrow = c(2, 2))
```

```r
acf(mcmc_beta[, 1], main = "ACF Beta 0",lag.max = 50000)
acf(mcmc_beta[, 2], main = "ACF Beta 1",lag.max = 50000)
acf(mcmc_beta[, 3], main = "ACF Beta 2",lag.max = 50000)
acf(mcmc_alpha, main = "ACF Alpha",lag.max = 50000)


# Lauching 5 Chains  ----------------------------------------------#

burnin <- 5000
n_chains <- 5
chains <- vector("list", n_chains)

for (i in 1:n_chains) {
  cat("Running chain", i, "\n")
  start_i <- starting_values(i)
  chains[[i]] <- run_mcmc(
    starting_values = start_i,
    M = M,
    y = y,
    x1 = x1,
    x2 = x2,
    hospital = hospital,
    sd_beta = c(0.062, 0.00115, 0.08),
    sd_u = 0.56
  )
}

# Traceplots


mcmc_chains_full <- lapply(chains, function(res) {
  mcmc(cbind(
    beta0 = res$beta_post[, 1],
    beta1 = res$beta_post[, 2],
    beta2 = res$beta_post[, 3],
    alpha = res$alpha_post
  ))
})
multi_chain_full <- mcmc.list(mcmc_chains_full)

par(mfrow = c(2, 2))
traceplot(multi_chain_full, col = 1:n_chains)


# Gelman-Rubin on first 10000


mcmc_chains_early <- lapply(chains, function(res) {
  mcmc(cbind(
    beta0 = res$beta_post[1:10000, 1],
    beta1 = res$beta_post[1:10000, 2],
    beta2 = res$beta_post[1:10000, 3],
    alpha = res$alpha_post[1:10000]
  ))
})

multi_chain_early <- mcmc.list(mcmc_chains_early)

gelman_early <- gelman.diag(multi_chain_early, autoburnin = FALSE)
print(gelman_early)
```

```r

gelman.plot(multi_chain_early, autoburnin = FALSE)


# Cut Burn-in

mcmc_chains <- lapply(chains, function(res) {
  mcmc(cbind(
    beta0 = res$beta_post[-(1:burnin), 1],
    beta1 = res$beta_post[-(1:burnin), 2],
    beta2 = res$beta_post[-(1:burnin), 3],
    alpha = res$alpha_post[-(1:burnin)]
  ))
})

multi_chain <- mcmc.list(mcmc_chains)
mcmc_chains_all <- lapply(chains, function(res) {
  v_names <- paste0("v_", 1:ncol(res$v_post))
  mcmc(cbind(
    beta0 = res$beta_post[-(1:burnin), 1],
    beta1 = res$beta_post[-(1:burnin), 2],
    beta2 = res$beta_post[-(1:burnin), 3],
    alpha = res$alpha_post[-(1:burnin)],
    setNames(as.data.frame(res$v_post[-(1:burnin), ]), v_names)
  ))
})
multi_chain_all <- mcmc.list(mcmc_chains_all)

# Traceplot after burn-in

par(mfrow = c(2, 2))
traceplot(multi_chain, col = 1:5)


# Final diagnostics

gelman_res <- gelman.diag(multi_chain, autoburnin = FALSE)
print(gelman_res)

gelman.plot(multi_chain, autoburnin = FALSE)

geweke_res <- lapply(multi_chain, geweke.diag)

for (i in seq_along(multi_chain)) {
  geweke.plot(multi_chain[[i]])
}

ess <- effectiveSize(multi_chain)
print(ess)

##############################################################################
# Multichain MCMC with extreme settings ---------------------------------------#
##############################################################################
# # We commented this section to avoid running it by default
# # as it is computationally intensive and not necessary for the main analysis.
n_chains <- 4
chains_extreme <- vector("list", n_chains)

# # ----------------------------
# # Starting values for chains
```

```r
# # ----------------------------
#
starting_values <- function(starting_index) {
   beta_list <- list(
     c(0, 0, 0),              # Chain 1: neutral
     c(5, 0.5, 1),            # Chain 2: strongly positive
     c(-5, -0.5, -1),         # Chain 3: strongly negative
     c(2, -0.8, 0.3)          # Chain 4: mixed
   )

   eta_list <- list(
     log(1),     # alpha = 1
     log(10),    # alpha = 10
     log(0.1),   # alpha = 0.1
     log(5)      # alpha = 5
   )

   list(
     beta = beta_list[[starting_index]],
     u  = eta_list[[starting_index]],
     v  = rep(1, 30)
   )
}

# # ----------------------------
# # Run MCMC chains
# # ----------------------------
#
for (i in 1:n_chains) {
  cat("Running chain", i, "\n")

  start_i <- starting_values(i)

  chains_extreme[[i]] <- run_mcmc(
    starting_values = start_i,
    M = M,
    y = y,
    x1 = x1,
    x2 = x2,
    hospital = hospital,
    sd_beta = c(0.062, 0.00115, 0.08),sd_u = 0.56
  )
}
burnin <- 0
# # Create mcmc objects from samples
mcmc_chains <- lapply(chains_extreme, function(res) {
  mcmc(cbind(
    beta0 = res$beta_post[-(1:burnin), 1],
    beta1 = res$beta_post[-(1:burnin), 2],
    beta2 = res$beta_post[-(1:burnin), 3],
    alpha = res$alpha_post[-(1:burnin)]
))
})

multi_chain <- mcmc.list(mcmc_chains)
# Traceplots before burn-in
par(mfrow = c(2, 2))
traceplot(multi_chain, col = 2:5)
```

```r

################################################################################
# QUESTION 4 ------------------------------------------------------------------#
################################################################################

################################################################################
#(a) Summary of our model parameter ------------------------------------------#
################################################################################

# --- B coefficients ---

beta_mcmc <- mcmc(beta_post)
print(summary(beta_mcmc))
print(HPDinterval(beta_mcmc, prob = 0.95))

# --- Alpha ---

alpha_mcmc <- mcmc(alpha_post)
print(summary(alpha_mcmc))
print(HPDinterval(alpha_mcmc, prob = 0.95))

# --- Random effects (Vg) ---

#CI
post_interv_v <- apply(v_post, 2, quantile, probs = c(0.025, 0.5, 0.975))
median_global_v <- median(as.vector(v_post))


# Densities of parameters

par(mfrow = c(1, 1))

# Beta0
ggplot(data.frame(beta0 = beta_post[, 1]), aes(x = beta0)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
  geom_density(color = 'blue') +
  labs(title = 'Density of Beta0', x = 'Beta0', y = 'Density') +
  theme_minimal(base_size = 12)

# Beta1
ggplot(data.frame(beta1 = beta_post[, 2]), aes(x = beta1)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
  geom_density(color = 'blue') +
  labs(title = 'Density of Beta1', x = 'Beta1', y = 'Density') +
  theme_minimal(base_size = 12)

# Beta2
ggplot(data.frame(beta2 = beta_post[, 3]), aes(x = beta2)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
  geom_density(color = 'blue') +
  labs(title = 'Density of Beta2', x = 'Beta2', y = 'Density') +
  theme_minimal(base_size = 12)

# Alpha
ggplot(data.frame(alpha = alpha_post), aes(x = alpha)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
```

```r
  geom_density(color = 'blue') +
  labs(title = 'Density of Alpha', x = 'Alpha', y = 'Density') +
  theme_minimal(base_size = 12)

# v_g

plot(1:G, xlim = range(post_interv_v), ylim = c(0.5, G + 0.5),
     type = "n", yaxt = "n", ylab = "Hospital index", xlab = "Random effect (v_g)",
     main = "95% Credible Intervals for v_g")
axis(2, at = 1:G, labels = 1:G)

for (g in 1:G) {
  lines(post_interv_v[c(1, 3), g], rep(g, 2), lwd = 2)
  points(post_interv_v[2, g], g, pch = 20)
}
abline(v = median_global_v, col = "red", lwd = 2, lty = 2)



################################################################################
#(b) Posterior predictive performance ----------------------------------------#
################################################################################

# --- Extraction ---
all_samples_mat <- do.call(rbind, lapply(multi_chain_all, as.matrix))
beta_post  <- all_samples_mat[, c("beta0", "beta1", "beta2")]
alpha_post <- all_samples_mat[, "alpha"]
v_post     <- all_samples_mat[, grep("^v_", colnames(all_samples_mat))]

M <- nrow(beta_post)
n <- length(y)

# --- Predictions ---
y_pred <- matrix(NA, nrow = M, ncol = n)
for (m in 1:M) {
  mu <- v_post[m, hospital] * exp(beta_post[m, 1] + beta_post[m, 2] * x1 + beta_post[m, 3] * x2)
  y_pred[m, ] <- rpois(n, mu)
}
yrep_vec <- as.vector(y_pred)

#Numerical comparaison y vs y_pred
print(summary(y))
summary(as.vector(y_pred))

# --- Visual comparaison ---
df_obs <- data.frame(y = y, type = "Observed")
df_pred <- data.frame(y = yrep_vec, type = "Predicted")
df_densite <- rbind(df_obs, df_pred)

df_tab <- table(df_densite$type, df_densite$y)
df_freq <- as.data.frame(df_tab)
colnames(df_freq) <- c("type", "y", "count")

df_freq$y <- as.numeric(as.character(df_freq$y))

freq_by_type <- tapply(df_freq$count, df_freq$type, sum)
df_freq$freq <- mapply(function(t, c) c / freq_by_type[[t]], df_freq$type, df_freq$count)

```

```r

df_obs <- df_freq[df_freq$type == "Observed", ]

p1 <- ggplot(df_obs, aes(x = factor(y), y = freq)) + geom_col(fill = "steelblue", color = "black") +
  labs(title = "Observed", x = "Number of visits", y = "Frequency") + theme_minimal(base_size = 13) +theme(plot.title = element_text(hjust = 0.5))+
  scale_x_discrete(limits = as.character(0:16))

p1

df_pred <- df_freq[df_freq$type == "Predicted", ]

p2 <- ggplot(df_pred, aes(x = factor(y), y = freq)) + geom_col(fill = "tomato", color = "black") +
  labs(title = "Predicted", x = "Number of visits", y = "Frequency") +
  theme_minimal(base_size = 13) +theme(plot.title = element_text(hjust = 0.5)) + scale_x_discrete(limits = as.character(0:16))
p2

p1 / p2

ggplot(df_freq[df_freq$y <= 16, ], aes(factor(y),freq, fill = type)) +geom_col(position = "identity", alpha = 0.5, color = "black") +
  scale_fill_manual(values = c("Observed" = "steelblue", "Predicted" = "tomato")) +
  labs(x = "Number of visits", y = "Frequency", fill = "Data type") + theme_minimal(base_size = 13)+scale_x_discrete(limits = as.character(0:32))+theme(
    legend.position = "top"
  )

##############################################################################
# QUESTION 5 ----------------------------------------------------------------#
##############################################################################


##############################################################################
#(a) JAGS Implementation ---------------------------------------------------#
##############################################################################


require(rjags)
require(R2WinBUGS)

ModelQ5 <- function() {

  # Likelihood
  for (i in 1:N) {
    y[i] ~ dpois(mu[i])
    mu[i] <- v[hospital[i]] * exp(beta0 + beta1 * x1[i] + beta2 * x2[i])
  }

  # Priors for regression coefficients
  beta0 ~ dnorm(0, 0.01)
  beta1 ~ dnorm(0, 0.01)
  beta2 ~ dnorm(0, 0.01)

  # Random effects per hospital
  for (g in 1:G) {
    v[g] ~ dgamma(alpha, alpha)
```

```r
  }

  # Hyperprior on alpha
  alpha ~ dgamma(0.01, 0.01)
}

model.file <- "question5_model.bug"
write.model(ModelQ5, model.file)

mydata <- list(y = y,x1 = x1,x2 = x2,hospital = hospital,N = length(y),G = 30)

myinits <- vector("list", 5)
for (i in 1:5) {
  start_i <- starting_values(i)
  myinits[[i]] <- list(
    beta0 = start_i$beta[1],
    beta1 = start_i$beta[2],
    beta2 = start_i$beta[3],
    alpha = exp(start_i$u),
    v = start_i$v
  )
}

jags_model <- jags.model(
  file = model.file,
  data = mydata,
  inits = myinits,
  n.chains = 5
)

samples <- coda.samples(
  model = jags_model,
  variable.names = c("beta0", "beta1", "beta2", "alpha", "v"),
  n.iter = 45000
)

samples_matrix <- as.matrix(samples) # For latter plots

# Numerical Summary
print(summary(samples))

# Graphical Summary

# v_g
v_post_jags <- as.matrix(samples)[, grep("^v", colnames(as.matrix(samples)))]

post_interv_v <- apply(v_post_jags, 2, quantile, probs = c(0.025, 0.5, 0.975))

median_global_v <- median(as.vector(v_post_jags))

par(mfrow = c(1, 1))
plot(1:G, xlim = range(post_interv_v), ylim = c(0.5, G + 0.5),
     type = "n", yaxt = "n", ylab = "Hospital index", xlab = "Random effect (v_g)",
     main = "95% CI for hospital random effects (JACGS)")

axis(2, at = 1:G, labels = 1:G)

for (g in 1:G) {
  lines(post_interv_v[c(1, 3), g], rep(g, 2), lwd = 2)
  points(post_interv_v[2, g], g, pch = 20)
```

```r
}

abline(v = median_global_v, col = "red", lwd = 2, lty = 2)


# Beta0
ggplot(data.frame(beta0 = samples_matrix[, "beta0"]), aes(x = beta0)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
  geom_density(color = 'blue') +
  labs(title = 'Density of Beta0', x = 'Beta0', y = 'Density') +
  theme_minimal(base_size = 12)

# Beta1
ggplot(data.frame(beta1 = samples_matrix[, "beta1"]), aes(x = beta1)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
  geom_density(color = 'blue') +
  labs(title = 'Density of Beta1', x = 'Beta1', y = 'Density') +
  theme_minimal(base_size = 12)

# Beta2
ggplot(data.frame(beta2 = samples_matrix[, "beta2"]), aes(x = beta2)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
  geom_density(color = 'blue') +
  labs(title = 'Density of Beta2', x = 'Beta2', y = 'Density') +
  theme_minimal(base_size = 12)

# Alpha
ggplot(data.frame(alpha = samples_matrix[, "alpha"]), aes(x = alpha)) +
  geom_histogram(aes(y=after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
  geom_density(color = 'blue') +
  labs(title = 'Density of Alpha', x = 'Alpha', y = 'Density') +
  theme_minimal(base_size = 12)


################ - Q6 - JAGS with new prior ########################

# DATA
#HospitalVisits <- read.delim("HospitalVisits.txt")
#y <- HospitalVisits$y           # Target variable (number of visits)
#x1 <- HospitalVisits$age        # Age of individuals
#x2 <- HospitalVisits$chronic    # Binary indicator of chronic disease
#hospital <- HospitalVisits$hospital  # Hospital ID for each individual
#G <- 30                         # Number of hospitals

# --- Libraries Needed ---
library(ggplot2)
library(mvtnorm)
library(coda)
library(rjags)
library(R2WinBUGS)
library(R2jags)

#Model function for JAGS
set.seed(123) #For reproducibility
ModelQ6 <- function() {
  for (i in 1:N) {
```

```
    y[i] ~ dpois(mu[i])
    mu[i] <- exp(z[hospital[i]] + beta0 + beta1 * x1[i] + beta2 * x2[i])
  }

  # Priors for regression coefficients
  beta0 ~ dnorm(0, 0.01)
  beta1 ~ dnorm(0, 0.01)
  beta2 ~ dnorm(0, 0.01)

  # Random intercepts per hospital
  for (g in 1:G) {
    z[g] ~ dnorm(0, tau)
  }

  # Hyperprior on tau
  tau ~ dgamma(0.01, 0.01)
}

# Save in a bug file
model.file <- "question6_model.bug"
write.model(ModelQ6, model.file)

#Data for JAGS
mydata <- list(
  y = y,
  x1 = x1,
  x2 = x2,
  hospital = hospital,
  N = length(y),
  G = 30
)

# Initializing values

myinits <- vector("list", 5) #with posterior mode for beta's
for (i in 1:5) {
  start_i <- starting_values(i)
  myinits[[i]] <- list(
    beta0 = start_i$beta[1],
    beta1 = start_i$beta[2],
    beta2 = start_i$beta[3],
    tau = 1,
    z = rep(0, mydata$G)
  )
}


set.seed(123)
#JAGS model construction
jags_model <- jags.model(
  file = model.file,
  data = mydata,
  inits = myinits,
  n.chains = 5,  n.adapt = 5000
)
set.seed(123)
update(jags_model, n.iter = 5000)


set.seed(123)
```

```r
#Sampling posterior
samples <- coda.samples(
  model = jags_model,
  variable.names = c("beta0", "beta1", "beta2", "tau", "z"),
  n.iter = 45000
)

#Summary posterior distribution
print(summary(samples))

#Traceplots for Beta's and Tau

coda :: traceplot ( samples[, c("beta0", "beta1", "beta2", "tau")] )

#Gelman-Rubin R
gelman.diag (samples )


obj1 = samples [[ 1 ]]
obj2 = samples [[ 2 ]]
obj3 = samples [[ 3 ]]
obj4 = samples [[ 4 ]]
obj5 = samples [[ 5 ]]

#Geweke
geweke.diag ( samples )


# ESS measures
combined_chain = rbind ( obj1 , obj2 , obj3, obj4, obj5 )
effectiveSize(combined_chain)

#HPD Intervals
combined_mcmc <- coda::mcmc(combined_chain)
HPD_ints <- HPDinterval(combined_mcmc, prob = 0.95)
print(HPD_ints)

#Credible intervals for v_g
z_cols <- colnames(combined_mcmc[, 5:34])
z_samples <- combined_mcmc[, z_cols]
v_samples <- exp(as.matrix(z_samples))

post_credint_v <- apply(v_samples, 2, quantile, probs = c(0.025, 0.5, 0.975))
median_global_v <- median(as.vector(v_samples))

par(mfrow = c(1,1))
plot(1:G, xlim = range(post_credint_v), ylim = c(0.5, G+0.5), type = "n",
     yaxt = "n", ylab = "Hospital index", main = "CI 95% for hospital random effects (JAGS)")
axis(2, at = 1:G, labels = 1:G)
for (g in 1:G) {
  lines(post_credint_v[c(1,3), g], rep(g, 2), lwd = 2)
  points(post_credint_v[2, g], g, pch = 20)
}
abline(v = median_global_v, col = "red", lwd = 2, lty = 2)

# Density + Histogram of tau
ggplot(data.frame(tau = combined_mcmc[, 4]), aes(x = combined_mcmc[, 4])) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = 'lightgray', color = 'black', alpha = 0.7) +
```

```r
  geom_density(color = 'blue') +
  labs(title = 'Density of Tau', x = 'Tau', y = 'Density') +
  theme_minimal(base_size = 12)

#Predictions for new model

b0_samples <- combined_mcmc[, "beta0"]
b1_samples <- combined_mcmc[, "beta1"]
b2_samples <- combined_mcmc[, "beta2"]

n <- length(y)
n_iter <- 5000
y_rep_matrix <- matrix(NA, nrow = n_iter, ncol = n)
set.seed(123)
for (m in 1:n_iter) {
  beta0 <- b0_samples[m]
  beta1 <- b1_samples[m]
  beta2 <- b2_samples[m]
  z_m <- as.numeric(z_samples[m, ])

  mu <- exp(z_m[hospital] + beta0 + beta1 * x1 + beta2 * x2)
  y_rep_matrix[m, ] <- rpois(n, lambda = mu)
}



y_rep_vec <- as.vector(y_rep_matrix) #Obtain the values in a vector



#Combined histograms
set.seed(123) #Set seed to reproduce results
y_rep_sample <- sample(y_rep_vec, size = 10000, replace = FALSE) #Sufficent number of values to be precise, considering all values is computationally expensive

df_obs <- data.frame(valeur = y, type_element = "Observed") #df with y
df_pred <- data.frame(valeur = y_rep_sample, type_element = "Predicted") #df with predicted y's
df_combined <- rbind(df_obs, df_pred)
ggplot() +
  geom_histogram(data = df_obs, aes(x = valeur, y=after_stat(density), fill = type_element),
                 binwidth = 1, boundary = -0.5, alpha = 0.6, color = "black") +
  geom_histogram(data = df_pred, aes(x = valeur, y=after_stat(density), fill = type_element),
                 binwidth = 1, boundary = -0.5, alpha = 0.6, color = "black") +
  scale_fill_manual(name = "Data type", values = c(
    "Observed" = "skyblue",
    "Predicted" = "salmon"
  )) +
  labs(
    x = "Number of visits",
    y = "Density"
  ) +
  theme_minimal()

#Individual plots
df_obs_ind <- data.frame(valeur = y, type = "Observed")
df_pred_ind <- data.frame(valeur = y_rep_sample, type = "Predicted")
df_combined_ind <- rbind(df_obs_ind, df_pred_ind)
```

```r

ggplot(df_combined_ind, aes(x = valeur, fill = type)) +
  geom_histogram(aes(y=after_stat(density)), binwidth = 1, boundary = -0.5, color =
  ↪  "black", alpha = 0.8) +
  facet_wrap(~ type, ncol = 1) +
  scale_fill_manual(values = c("Observed" = "steelblue", "Predicted" = "tomato")) +
  theme_minimal() +
  labs(,
       x = "Number of visits",
       y = "Density"
  )
#Summary statistics for observed vs. predicted
y_bar <- mean(y)
y_pred_bar <- mean(y_rep_sample)
sd_y <- sd(y)
sd_y_pred <- sd(y_rep_sample)
Q1_y <- quantile(y, 0.25, na.rm = TRUE)
Q3_y<- quantile(y, 0.75, na.rm = TRUE)
Q1_y_pred <- quantile(y_rep_sample, 0.25, na.rm = TRUE)
Q3_y_pred<- quantile(y_rep_sample, 0.75, na.rm = TRUE)
median_y <- median(y, na.rm = TRUE)
median_y_pred <- median(y_rep_sample, na.rm = TRUE)
IQR_y <- Q3_y - Q1_y
IQR_y_pred <- Q3_y_pred - Q1_y_pred

mean(v_samples) #posterior mean for v_g
median(v_samples) #median
min(v_samples)
max(v_samples)
```